



A novel clustering approach for recommendation systems using adaptive fuzzy clustering with Jensen–Shannon divergence

Gökhan Kayhan^{ID*}, Naciye Aydin^{ID}, Sercan Demirci^{ID}

Ondokuz Mayıs University, Department of Computer Engineering, Kurupelit Campus, Samsun, 55139, Atakum, Türkiye

ARTICLE INFO

Keywords:

Fuzzy clustering
Fuzzy C-Means
Jensen Shannon
Interest metric
Movie recommendation system

ABSTRACT

In today's world, where users are surrounded by a multitude of products, recommender systems are employed to assist users in finding products of interest. Clustering methods are frequently utilized in recommender systems to suggest relevant products. Fuzzy clustering techniques, one of the most commonly used clustering methods, determine the degree of relevance of each product to a cluster through the membership matrix it generates. However, determining the number of clusters in these methods poses a challenge. This study proposes an Adaptive Fuzzy C-Means Jensen Shannon (AFCM-JS) algorithm, a fuzzy and interest-based clustering method that estimates the number of clusters. The proposed AFCM-JS algorithm is implemented on an artificial dataset consisting of 6 clusters and 1000 elements. The results of the study are compared with Fuzzy C-Means (FCM), Probabilistic C-Means (PCM), and Probabilistic Fuzzy C-Means (PFCM) methods, which are fuzzy-based clustering algorithms, and the interest-based method JS. To evaluate the comparison results, 7 different cluster validity indices and an accuracy metric are employed. AFCM-JS method consistently and accurately predicted the number of clusters when tested with different maximum cluster numbers. When the clustering ability of the method is tested with cluster validity indices and the accuracy metric, AFCM-JS is found to be successful. The performance of the AFCM-JS method is tested on a dataset created for a movie recommendation system with the aim of recommending movies to users. For this purpose, movie data is weighted with a Dirichlet function for action, adventure, comedy, drama, and horror genres, creating a dataset that includes the characteristics of these 5 movie genres. The AFCM-JS method is compared with 3 different fuzzy clustering methods using 7 different cluster validity indices with this created movie dataset. Additionally, the AFCM-JS algorithm is compared with the other 3 fuzzy clustering methods based on the accuracy metric. As a result of this comparison, the AFCM-JS method achieves the highest performance among the methods with 81.9366%. Furthermore, when the performance of the proposed method is compared in terms of cluster validity indices, the AFCM-JS method successfully predicts the appropriate number of clusters and effectively groups similar movies according to their genres, accomplishing the purpose.

1. Introduction

The exponential growth in the volume of information accessible through the internet in our contemporary times has given rise to the issue of excessive information load, thereby leading users to encounter challenges in locating the products of their need or interest. The confusion caused by an excessive load of information has necessitated the development of recommendation systems in various fields such as movies, music, documentaries, books, e-commerce, e-learning, news, health, and more.

The purpose of recommendation systems is to provide users with predictions or suggestions about relevant products. Prediction estimates the rating a user would give to a product and provides a probability

of whether the user would be interested in that product. On the other hand, recommendation refers to suggesting a list of products that the user is likely to enjoy. With the huge increase in data in recent years recommendation system studies are being conducted in many areas where they may be needed. In literature, recommendation system techniques related to recommendation types are divided into different categories in different studies. Many studies examine recommendation systems in three techniques: content-based filtering, collaborative filtering, and hybrid filtering [1,2]. Some studies also refer different types of recommendation systems, such as demographic filtering methods [3] and knowledge-based recommendation systems [4]. In the study by Son and Kim, a novel content-based filtering (CBF) method using a multi-attribute network to effectively calculate relationships between items

* Corresponding author.

E-mail address: gkayhan@omu.edu.tr (G. Kayhan).

<https://doi.org/10.1016/j.csi.2025.104035>

Received 13 May 2024; Received in revised form 1 April 2025; Accepted 9 June 2025

Available online 2 July 2025

0920-5489/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

was proposed, incorporating centrality and clustering techniques to enhance recommendation performance [5]. Gazdar and Hidri developed a similarity measure for collaborative filtering using mathematical tools such as integral equations, linear differential equation systems, and non-linear systems, to improve the accuracy of recommendations [6]. Cui et al. focused on the time effect in recommendation systems in their study. They developed a method to measure changes in a user's interest over time using the time correlation coefficient [7]. Walek and Fojtik conducted a hybrid movie recommendation study by combining collaborative filtering with a content-based approach and fuzzy expert systems [8]. Aggarwal et al. developed a collaborative filtering-based movie recommendation system that designs movie rankings based on user interest and movie popularity [9].

In addition to these studies, clustering-based recommendation systems have been widely explored in the literature as a significant approach that aims to enhance recommendation accuracy by grouping users or items based on their similarities. Zahra et al. presented a k-means clustering-based recommendation algorithm in their study, addressing scalability issues in traditional recommender systems and proposing a more efficient centroid selection method to mitigate the negative effects of random centroid selection on accuracy and cost [10]. Vara et al. evaluated the k-means clustering algorithm to enhance the recommendation accuracy of the RICEST Journal Finder System, analyzing over 15,000 articles and comparing system recommendations before and after clustering, with results demonstrating an increase in the target journal's top-three ranking rate from 40% to 80%, thereby proving the significant impact of clustering on recommendation accuracy [11]. Mann and Chawla presented a hybrid clustering algorithm that combines partition-based and hierarchical clustering techniques to recommend the nearest passenger pickup locations for cab drivers, demonstrating its superior clustering performance compared to K-Means and BIRCH algorithms [12]. Li and Kim proposed a clustering-based approach for hybrid recommender systems, which aim to improve recommendation accuracy by combining different filtering techniques [13]. Gulzar et al. investigated the Ordered Clustering-based Algorithm (OCA) to enhance e-commerce recommendation systems by addressing cold-start and data sparsity issues through collaborative filtering, demonstrating superior performance in Precision, Recall, and F-measure compared to previous approaches [14]. Guo et al. suggested a multiview clustering approach for recommender systems that integrates rating-based similarity and social trust to improve clustering accuracy and coverage [15]. Khoo conducted a study on the relationship between the way streaming services like Netflix recommend content to users and the diversity of content [16]. Rashidi et al. introduced a new method for clustering users with different preferences in recommendation systems [17]. Koohi and Kiani designed a recommendation system in their study using content-based filtering that utilizes genre correlation. In their research, they conducted a movie recommendation system study for the first time using the Fuzzy C-Means (FCM) algorithm in a user-based collaborative filtering method [18].

The proposed method has been used for grouping similar movie genres in a movie recommendation system. The main contributions of this study can be summarized as follows:

- The developed method integrates the Fuzzy C-Means (FCM) algorithm with the Jensen-Shannon (JS) distance metric. This hybrid approach aims to enhance the estimation of the optimal number of clusters by leveraging the strengths of both techniques.
- Experimental results indicate that the AFCM-JS method is effective in accurately estimating the number of clusters. The method consistently produced reliable results across various datasets and scenarios.
- When compared with other established clustering methods, the AFCM-JS approach demonstrated superior performance. The comparative analysis highlighted its strengths in terms of accuracy and robustness.

- The AFCM-JS method shows particular promise in applications such as movie recommendation systems. In such systems, where capturing and utilizing interest relationships between users is crucial, the method proved to be particularly effective in identifying the optimal number of clusters.

The remainder of this paper is structured as follows: Section 2 provides background information. Section 3 defines the problem in detail. Section 4 presents the proposed method. Section 5 explains the experimental results and evaluation. Section 6 discusses the conclusions and potential future work.

2. Background

2.1. Fuzzy C-Means algorithm

The Fuzzy C-Means (FCM) Algorithm is a fuzzy clustering algorithm based on the principle of fuzzy logic developed by [19] that uses a goal function. Each data sample has a membership value that can belong to multiple clusters. The sum of membership values for different groups must be 1. The objective function of the FCM algorithm is given in Eq. (1). In this equation, U represents the membership matrix, V represents the cluster centers, N represents the number of data, k represents the number of clusters within the range of $2 \leq k \leq n$, U_{ij} represents the probability of (i, j) data belonging to the k th cluster, d_{ij} represents the Euclidean distance between the center of the k th cluster and the data point, and m represents the fuzziness parameter with a value in the range of $(1 \leq m < \infty)$.

$$J(U, V) = \sum_{i=1}^n \sum_{j=1}^k U_{ij}^m d_{ij}^2 \quad (1)$$

The calculation of the cluster centers is given in Eq. (2). In this equation, U_{ik} is the membership value of the i th data sample to the k th cluster. The sum of the membership of the i th data sample to all clusters is equal to 1. X_{ij} represents the data point.

$$V_{ij} = \frac{\sum_{i=1}^N U_{ij}^m X_{ij}}{\sum_{i=1}^N U_{ij}^m} \quad (2)$$

Eq. (3) gives the update of the membership matrix for values consisting of N data samples and k clusters.

$$U_{ij} = \frac{1}{\sum_{j=1}^k \left(\left(\frac{d_{ij}}{d_{ji}} \right)^{\frac{2}{m-1}} \right)} \quad (3)$$

Fig. 1 shows the flowchart of the FCM algorithm. FCM algorithm takes the number of clusters, fuzziness parameter, number of iterations, and threshold value as input parameters. The algorithm initialize with the creation of a random membership matrix. Then the cluster centers are calculated. Afterwards, the membership degree of each data element to a cluster is assigned. In each iteration, the cluster centers and the membership degrees are updated.

2.2. Jensen Shannon divergence

Jensen Shannon Divergence (JSD) is a measure of the similarity between two probability distributions. It is a symmetric and finite version of the Kullback-Leibler (KL) divergence, and it ranges from 0 to 1, where 0 indicates that the two distributions are identical, and 1 indicates that the two distributions are completely different [20]. Jensen-Shannon distance calculates a symmetric calculation using the KL divergence shown in Eq. (4).

$$KL(X \parallel Y) = \sum_{i=1}^N X_i \log \left(\frac{X_i}{Y_i} \right) \quad (4)$$

$$JSD(P \parallel M) = \frac{1}{2} KL(P \parallel M) + \frac{1}{2} KL(Q \parallel M) \quad (5)$$

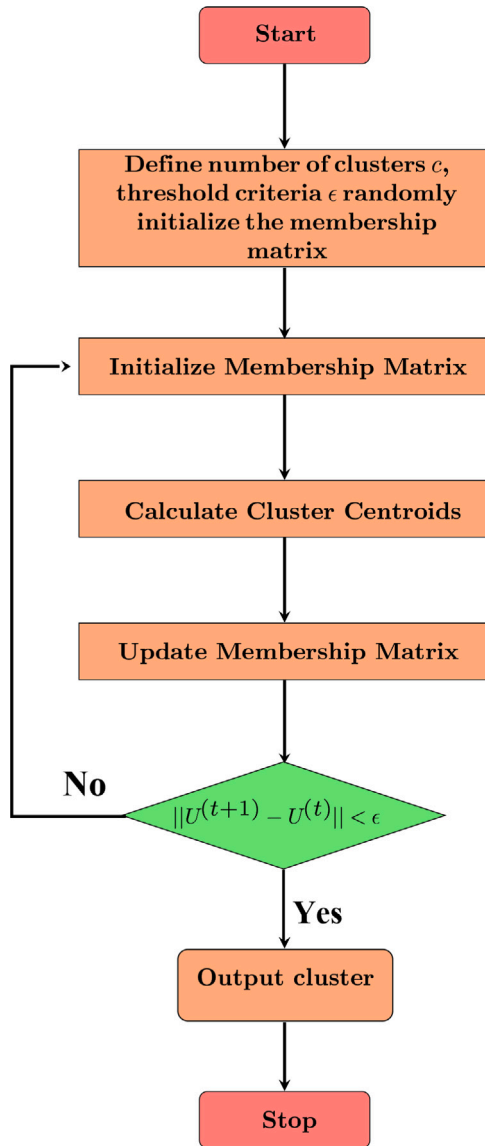


Fig. 1. The flowchart of the FCM algorithm.

$$M = \frac{1}{2}(Q + P) \quad (6)$$

$$JSD(P \parallel Q) = JSD(Q \parallel P) \quad (7)$$

JSD is calculated by taking the average of the KL divergence between the probabilities of P and Q and the probability of M. Eq. (6) represents the average of the P and Q values shown. An important feature of the Jensen–Shannon divergence is that the relationship between P and Q is equal to the relationship between Q and P probability. Eq. (7) expresses a symmetrical similarity measure that shows that the JSD of P and Q probability and the JSD of Q and P probability yield the same result. The probabilities of P and Q values must add up to 1. The Jensen–Shannon distance is calculated by calculating the square root of the JSD value shown in Eq. (5).

3. Problem definition

The FCM method, an unsupervised machine learning approach, clusters data based on membership degrees rather than strict assignments, making it distinct from classical clustering methods. Widely

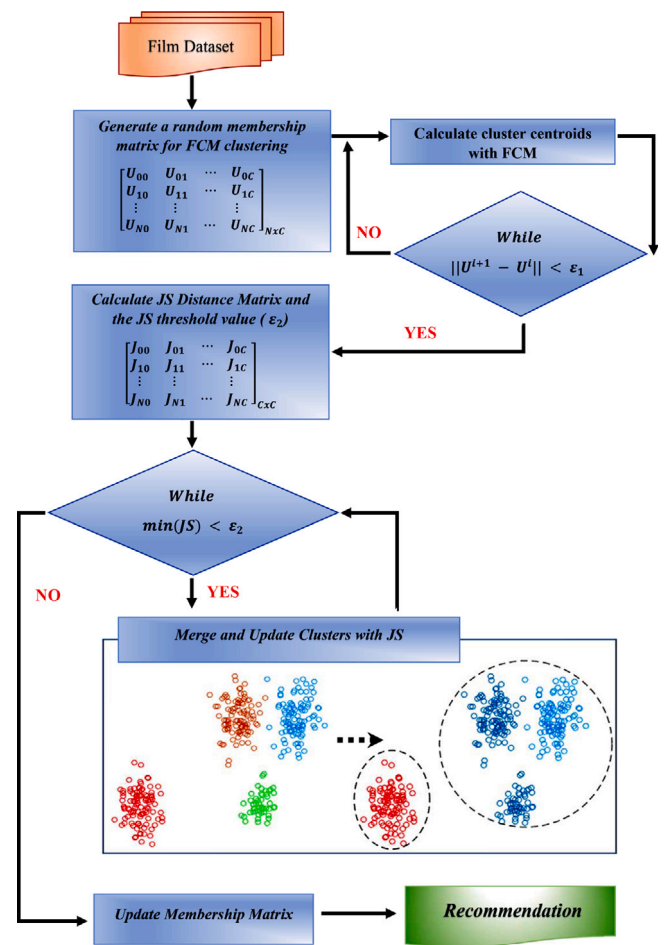


Fig. 2. Flowchart of the proposed model.

used in fields like recommendation systems, healthcare, bioinformatics, marketing, and finance, FCM requires specifying the correct number of clusters before execution. However, in real-world datasets, the complexity of data and the challenges in analysis make it difficult to determine the actual number of clusters. This article proposes the AFCM-JS algorithm, a fuzzy clustering method that dynamically updates the number of clusters based on an interest metric, addressing the issue of determining the number of clusters encountered in the FCM method. The developed AFCM-JS method is adapted to a movie dataset consisting of various movie genres, enabling the clustering of movies with unknown genre distributions based on their mutual interest.

The general framework of the AFCM-JS method adapted to the movie dataset is presented in Fig. 2. In the initial step of the method, the dataset is clustered using the FCM algorithm to obtain an $N \times C$ partition matrix (N: number of data points, C: number of clusters) with the maximum cluster numbers. Each column U_{ij} in the obtained partition matrix represents the degree to which each data element belongs to a particular cluster. Subsequently, a $C \times C$ Jensen Shannon distance matrix is computed, containing the Jensen Shannon distances between each pair of clusters. Threshold values are determined using the Interquartile Range (IQR) method based on the obtained Jensen Shannon distance matrix. Cluster centers and the number of clusters are then updated according to this threshold value, resulting in the formation of new clusters and cluster centers. Recommendations for movies are made based on the clustered similar genres.

4. Proposed method: AFCM-JS

The AFCM-JS algorithm is an enhanced algorithm developed to address the problem of determining the appropriate number of clusters in the FCM algorithm. The AFCM-JS algorithm consists of two main components. The first part involves running the FCM method with a maximum cluster numbers, while the second part involves generating the Jensen–Shannon distance matrix and determining the threshold value to create clusters based on an appropriate number. Algorithm 1 provides the pseudocode for the AFCM-JS algorithm. The following is an explanation of the pseudocode for the AFCM-JS algorithm:

- **Input-Output:** The input parameters required for the operation of the developed AFCM-JS algorithm and the outputs to be generated upon algorithm completion are determined. The sensitivity of the AFCM-JS algorithm to its parameters, particularly the fuzziness parameter and the maximum number of clusters, can indeed impact performance. For instance, the fuzziness parameter m , which controls the degree of membership distribution across clusters, has been fixed at a default value of 2 in this study, as this is a common value used in many fuzzy clustering applications. Upon examination of the input parameters, m : the fuzziness parameter, k : the maximum cluster numbers, l : the number of iterations parameter, and ε_1 : Stopping criterion threshold value for FCM. The outputs to be generated when the algorithm terminates are k_T : the estimated number of clusters, V' : cluster centers, and U' : the updated membership matrix, also known as the partition matrix.
- **Step 1–12:** Following the determination of the relevant parameters, Step 1 is executed by generating a random membership matrix. Subsequently, the steps of the FCM algorithm between 2 and 12 are executed to update cluster centers and the membership matrix. When cluster centers, obtained based on the maximum cluster numbers, and the membership matrix are computed, the first part of the AFCM-JS algorithm, which is the FCM method, is completed.
- **Step 13–14:** The number of columns in the membership matrix generated by the FCM method is equal to the maximum cluster numbers (k) specified at the beginning of the algorithm. The AFCM-JS algorithm estimates the appropriate number of clusters by utilizing the updated membership matrix obtained during the operation of the FCM algorithm. Among each pair of columns in the membership matrix, the Jensen Shannon (JS) distance, as depicted in Eq. (5), is computed, resulting in a $(k \times k)$ dimensional JS distance matrix. In Step 14, the smallest JS value in the obtained distance matrix is found.
- **Step 15:** The JS threshold value is determined to estimate the number of clusters. The JS threshold parameter indicates the level of interest between two clusters, in other words, how much relevance the clusters have to each other. The JS threshold value can be determined by running the algorithm with different threshold values randomly or by comparing the results in cluster validity indices. However, in this case, determining the JS value can be cumbersome and time-consuming. To determine the JS threshold value easily and accurately, the interquartile range (IQR) method is used. The interquartile range is the difference between the first quartile, which is the 25th percentile, and the third quartile, which is the 75th percentile, of a sorted data set. When the first quartile is denoted as $Q1$ and the third quartile as $Q3$, the interquartile range is calculated based on the difference between these two quartiles in Eq. (8). To determine the threshold value, the outlier detection formula provided in Eq. (9), which is based on the interquartile range, is used. The value T in Eq. (9) represents the minimum threshold value that can be used for the data set, while c represents a constant decided by the user, typically used as 1.5 [21].

$$IQ_R = Q3 - Q1 \quad (8)$$

$$T = Q1 - c \cdot IQ_R \quad (9)$$

- **Step 16–25:** In Step 15, the threshold value calculated (ε_2) is compared to the minimum JS value within the matrix. The algorithm progresses in two different ways depending on whether the (ε_2) value is greater or smaller than the minimum JS value obtained from the pair of clusters that achieves the minimum JS value. If the minimum JS value between cluster pairs is not smaller than the (ε_2) value, the algorithm terminates with the current clusters. As long as the calculated minimum JS value is smaller than the threshold value, the loop proceeds to the next step and continues.
 - **Step 17–19:** The number of elements in the two clusters with the minimum JS value within the matrix are compared. The cluster with a larger number of elements is selected as the cluster leader. The cluster center of the cluster with a larger number of elements is updated based on the weighted average of the centers of the two clusters. In Eq. (10), C_i represents the minimum cluster centers, and W_i represents the number of elements associated with the cluster centers.
- $$V_C^I = \frac{\sum_{i=1}^N W_i C_i}{W_i} \quad (10)$$
- **Step 20–22:** The cluster with fewer elements in the cluster pair is eliminated because it contributes its elements to the larger cluster. Thus, the current number of clusters decreases by one. The deleted cluster is also removed from the JS distance matrix. After removing the membership of the smaller cluster from the matrix, a new pair of clusters with the minimum JS value is calculated in the JS distance matrix.
 - **Step 23:** The matrix changes with the removal of the membership of the smallest cluster in the JS distance matrix. Therefore, just like in Step 15, the JS threshold value is recalculated.
 - **Step 25–27:** When the smallest JS value between clusters is not less than the JS threshold value (ε_2), the appropriate number of clusters is determined. Finally, the membership matrix (U') is updated with the determination of the number of clusters and the algorithm is terminated.

Theorem 4.1. The overall complexity of the Algorithm 1 is $O(l \cdot N \cdot k + k^2)$.

Proof. There are two phases of the algorithm. The first phase involves FCM Clustering. Initially, the FCM clustering phase uses nested loops. The outer loop (indexed by l) iterates up to the maximum number of iterations (l). The middle loop (indexed by i) iterates over the data points (N). The innermost loop (indexed by j) iterates over the cluster centers (k). Each loop performs constant time operations, resulting in a complexity of $O(l \cdot N \cdot k)$ for this step. The second phase involves calculating the Jensen–Shannon distance matrix, which requires a loop that takes $O(k^2)$ time. The while loop runs until the minimum Jensen–Shannon distance is greater than or equal to the Jensen–Shannon threshold (ε_2). Each iteration of the while loop involves operations to merge clusters and update cluster centers and memberships, which takes $O(N \cdot k)$ time in the worst case. $\square$

Theorem 4.2. There are at most $\max(k_T) = \left\lceil \frac{N}{\Delta_{lower}} \right\rceil$ estimated clusters in the AFCM-JS method.

Proof. Given a fuzzy clustering system where each cluster contains between Δ_{lower} and Δ_{upper} members, we aim to determine the maximum estimated number of clusters $\max(k_T)$. We assume that N be the data set length in the system, k_T be the estimated number of clusters, and each cluster i has n_i members, where $\Delta_{lower} \leq n_i \leq \Delta_{upper}$. To maximize the estimated number of clusters, each cluster should have the minimum number of members, which is Δ_{lower} . This is because if each cluster has more members, then fewer clusters can be formed. The data set

length is given by $N = \sum_{i=1}^{k_T} n_i$. Since each cluster i has at least Δ_{lower} members, the inequality becomes $N \geq k_T \cdot \Delta_{lower}$. From this inequality, we can solve for the maximum cluster numbers $k_T \leq \frac{N}{\Delta_{lower}}$. This represents the maximum cluster numbers possible when each cluster has exactly Δ_{lower} nodes. If each cluster had more than Δ_{lower} nodes, the number of clusters k_T would decrease because n_i would be greater than Δ_{lower} . Therefore, the maximum estimated number of clusters $\max(k_T) = \left\lfloor \frac{N}{\Delta_{lower}} \right\rfloor$. $\square$

Algorithm 1: AFCM-JS Algorithm

Input: k : Maximum cluster number, l : Maximum number of iterations, m : Fuzziness parameter, ε_1 : Stopping criterion threshold value for FCM and N : Data set length

Output: k_T : Estimated number of clusters, V' : Cluster centers and U' : Membership matrix

```

1 Initialize membership matrix ( $U^0$ ) randomly;
2 for  $t = 1$  to  $l$  do
3   for  $i = 1$  to  $N$  do
4     for  $j = 1$  to  $k$  do
5       Calculate cluster centers ( $V'$ ) and euclidean distance;
6       Update membership matrix ( $U^{i+1}$ );
7       if  $\|U^{i+1} - U^i\| \leq \varepsilon_1$  then
8         Exit all loops;
9       end
10    end
11  end
12 end
13 Calculate JS distance matrix;
14 Find the minimum value in the JS distance matrix (min JS( $x, y$ )) where  $x, y \in V'$ ;
15 Calculate the Jensen Shannon threshold value ( $\varepsilon_2$ );
16 while min JS( $x, y$ )  $< \varepsilon_2$  do
17   Identify the pair of clusters that provide the minimum JS value;
18   Compare the number of elements in the clusters obtained in Step 17;
19   Calculate the new cluster center ( $V'$ ) by merging the pair of clusters obtained in Step 17 using a weighted average of their centers;
20    $k = k - 1$ ;
21   Remove the membership of the smaller cluster from the JS distance matrix;
22   Recalculate the minimum value in the JS distance matrix;
23   Recalculate the Jensen Shannon threshold value ( $\varepsilon_2$ );
24 end
25  $k_T = k$ ;
26 Update membership matrix ( $U'$ );
27 return  $k_T, V', U'$ ;

```

5. Experimental evaluation

In this section, the proposed AFCM-JS method has been compared with FCM, PCM, and PFCM algorithms using two different datasets, one being a synthetic dataset and the other a real movie dataset. The 7 different cluster validity indices and accuracy metric (gMeasure) used in this comparison are explained.

5.1. Cluster validity measures and accuracy metric

Some metrics used in cluster validity index may exhibit a monotonic decreasing trend as the number of clusters increases. Additionally, cluster validity indices test cluster formations from different perspectives such as separation, compactness, or both. Therefore, using a

single performance measure is insufficient to explain the accuracy of a method. In this study, to demonstrate the accuracy of the proposed method, performance indices commonly preferred in fuzzy clustering methods such as Partition Entropy (PE), Partition Coefficient (PC), Modified Partition Entropy (MPE), Modified Partition Coefficient (MPC), Fukuyama–Sugeno (FS), SC, and Xie–Beni (XB) have been used.

- Partition Entropy (PE): Partition entropy is a scalar measure that quantifies the fuzziness in the partition matrix obtained after clustering. The partition entropy, denoted as PE, takes values within the range $[0, \log c]$, where c represents the number of clusters. PE value close to 0 indicates effective clustering, while values nearing the upper limit suggest a lower success in the method's ability to form clusters [22]. Bezdek proposed a validity index PE for fuzzy clustering in Eq. (11), where N is the number of samples and U represents the partition matrix [23].

$$PE(U, c) = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^c U_{ij} \log(U_{ij}) \quad (11)$$

- Partition Coefficient (PC): PC is a performance measure based on minimizing the binary fuzzy intersection in the partition matrix resulting from clustering. This performance measure expresses the membership sharing in the partition matrix as a numerical value. The PC value, denoted by c for the number of clusters, falls within the range $[1/c, 1]$. As the PC value approaches 1, clustering performance improves [22]. Bezdek proposed a validity index PC for fuzzy clustering in Equation (12), where N is the number of samples and U represents the partition matrix [24].

$$PC(U, c) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^c (U_{ij})^2 \quad (12)$$

- Modified Partition Entropy (MPE): MPE is an adaptation of the traditional Partition Entropy (PE) measure, designed to exhibit a more predictable, monotonic behavior. This metric is particularly useful in evaluating the quality of clustering results, especially in the context of fuzzy clustering, where data points can belong to multiple clusters with varying degrees of membership. The MPE value provides an indication of the level of disorganization in the clustering process. When the MPE value is close to 1, it suggests that the clustering is disorganized, meaning that the clusters are not well-defined or are overlapping significantly. Conversely, MPE values approaching 0 indicate a well-organized clustering, where the clusters are clearly distinct and well-separated [25,26]. The formula for MPE is given in Eq. (13).

$$MPE = 1 - \frac{PE}{\log c} \quad (13)$$

In this formula, c represents the number of clusters.

- Modified Partition Coefficient (MPC): The MPC is an adaptation of the traditional Partition Coefficient (PC) measure, developed to provide a more consistent and monotonic evaluation of clustering quality. This metric is particularly valuable in fuzzy clustering scenarios, where data points can belong to multiple clusters with varying degrees of membership [22]. The MPC value serves as an indicator of the clustering structure. Specifically, higher MPC values indicate better-defined clusters, meaning that the clusters are more distinct and data points have a stronger membership in a single cluster. Lower MPC values suggest more overlap and less distinction between clusters. The formula for MPC is given in Eq. (14).

$$MPC = \frac{c \cdot PC - 1}{c - 1} \quad (14)$$

In this formula, c denotes the number of clusters.

- **Fukuyama–Sugeno Index (FS):** The FS index is a metric developed to evaluate both the intra-cluster compactness and inter-cluster separation in clustering. It is designed to quantify how well-defined the clusters are and how well-separated they are from one another. A smaller FS value signifies better compactness within clusters and greater separation between clusters. As the number of clusters increases, the FS value generally decreases in a monotonic trend [27]. The formula for FS is given in Eq. (15).

$$FS(U, V; X) = \sum_{i=1}^c \sum_{j=1}^n (U_{ij})^m (\|X_j - V_i\|^2 - \|V_i - \bar{V}\|^2) \quad (15)$$

In this formula:

- $\|X_j - V_i\|^2$ measures the compactness of the i th cluster, where X_j represents a data point and V_i represents the centroid of the i th cluster. This term captures the squared distance between each data point and its corresponding cluster centroid.
- $\|V_i - \bar{V}\|^2$ indicates the degree of separation between the i th cluster and the mean of all cluster centroids, where $\bar{V}$ is calculated $\bar{V} = \frac{1}{n} \sum_{j=1}^n X_j$. Here, $\bar{V}$ is the average of all cluster centroids. This term reflects how distinct the i th cluster is from the overall mean of the centroids.

The FS index combines these two measures to provide an overall assessment of clustering quality. A lower FS value denotes clusters that are more compact and better separated from one another, which indicates more effective clustering.

- **Xie–Beni Index (XB):** The Xie–Beni index is a validity measure used to evaluate clustering performance by focusing on two key aspects: cluster compactness and cluster separation. In the XB index, the numerator represents the compactness of the clusters, while the denominator reflects the separation between clusters. A lower XB value indicates better clustering performance. However, a notable limitation of this index is that it tends to decrease monotonically as the number of clusters increases [27]. The formula for XB is given in Eq. (16).

$$XB(U, V; X) = \frac{\sum_{i=1}^c \sum_{j=1}^n U_{ij}^2 \cdot \|V_i - X_j\|^2}{n \cdot d_{min}^2} \quad (16)$$

In this formula:

- n represents the total number of data points.
- c denotes the number of clusters.
- U_{ij} is the fuzzy membership value of data point X_j in cluster i .
- V_i is the center of cluster i .
- X_j is the data point j .
- d_{min} is the minimum distance between any pair of cluster centers, defined as $d_{min} = \min_{i \neq j} \|V_i - V_j\|$.
- **SC index:** The SC index is a validity index that combines cluster compactness and separation. A higher SC value indicates better clustering. The SC index is calculated by determining the ratio of inter-cluster separation to intra-cluster compactness [28]. The mathematical expression of the SC index is provided in Eq. (17).

$$SC(U, V, X) = \frac{Sep(c)}{Comp(c)} \quad (17)$$

Sep(c) is the fuzzy separation of fuzzy clusters, calculated in Eq. (18).

$$Sep(c) = \text{trace}(S_B) \quad (18)$$

The matrix S_B given in Eq. (19) represents the between-cluster fuzzy scatter matrix.

$$S_B = \sum_{i=1}^c \sum_{j=1}^n U_{ji}^m \cdot (v_i - \bar{v}) \cdot (v_i - \bar{v})^T \quad (19)$$

The total compactness of the fuzzy c-partition, denoted as $Comp(c)$, is calculated in Eq. (20).

$$Comp(c) = \sum_{i=1}^c \text{trace}(\sum_i i) \quad (20)$$

The fuzzy covariance matrix, denoted as $\sum_i$, is calculated in Eq. (21).

$$\sum_i = \frac{\sum_{j=1}^N U_{ji}^m (X_j - V_i)(X_j - V_i)^T}{\sum_{j=1}^N (U_{ji}^m)} \quad (21)$$

- **Accuracy Metric (gMeasure):** gMeasure is an accuracy metric designed to assess the accuracy of measurement within and between groups. In this accuracy metric, a group leader is dynamically created for measuring the distance between groups [29].

$$1 - \frac{1}{N'} \sum_{i,j \in [1,c]} \sum_{\substack{x \in G_i \\ y \in G_j}} \frac{|(x, y) - (L_i, L_j)|}{x_1 y_1}, \quad N' = \sum_{i,j \in [1,c]} |G_i| \cdot |G_j| \quad (22)$$

The calculation of the accuracy metric is given in Eq. (22). Here,

- G_i and G_j represent the groups, and c represents the number of groups.
- L_i and L_j denote the leaders of groups G_i and G_j , respectively.
- The pair (x, y) represents the distance between two data points.
- The value N gives the sum of the product of the elements of groups G_i and G_j .

gMeasure takes values in the range [0-1]. Values close to 1 indicate a better intra-group and inter-group distribution, implying improved clustering accuracy.

5.2. Synthetic data set

The synthetic data generated to test the effectiveness of the proposed method consists of randomly distributed values scaled around a specific mean, with the data grouped around that center with a certain spread. The dataset X , consisting of n data points distributed in a d -dimensional space and is structured around c distinct cluster centers, is calculated in Eq. (23). Each cluster is generated from a normally distributed random variable and shifted according to a predefined center.

$$X = \{x_i \in \mathbb{R}^d \mid i = 1, 2, \dots, n \cdot c\} \quad (23)$$

The cluster centers are defined as $C = \{c_1, c_2, \dots, c_c\}$, and the data points in each cluster are formulated in Eq. (24), where k is a scaling factor, c_j represents the center of cluster j , defined in $\mathbb{R}^d$, $\mathcal{N}(0, 1)$ denotes a normal distribution with zero mean and unit variance.

$$X_j = \frac{\mathcal{N}(0, 1)}{k} + c_j, \quad j \in \{1, 2, \dots, c\} \quad (24)$$

Thus, the synthetic dataset is obtained in Eq. (25):

$$X = \bigcup_{j=1}^c X_j \quad (25)$$

This formulation ensures that the dataset consists of non-overlapping clusters positioned across all d dimensions, preserving the underlying structure of the data. Using this method, a synthetic data set consisting of 1000 data samples, 6 different clusters, and 2 different features has been generated. The distribution of this synthetic data set is shown in Fig. 3.

The estimated cluster numbers when the synthetic data set with 6 clusters, shown in Fig. 2, is run through the AFCM-JS method are provided in Table 1. To demonstrate the consistency in estimated cluster numbers, the AFCM-JS method has been executed with 6 different

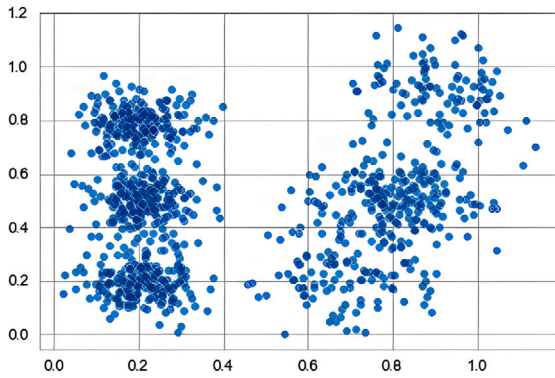


Fig. 3. Distribution of the synthetic data.

Table 1
Estimated cluster numbers with the AFCM-JS method for the synthetic data.

Maximum cluster numbers	Estimated cluster numbers
10	6
12	6
20	6
25	5, 6
50	6
100	6

maximum cluster numbers as shown in Table 1. In this table, it can be observed that when the AFCM-JS method is run with 5 different maximum cluster numbers: 10, 12, 20, 50, and 100, the estimated cluster numbers match the true cluster number. When the maximum cluster numbers was set to 25, the predicted number of clusters varied between 5 and 6. However, the AFCM-JS algorithm has provided a prediction close to the true cluster number for this dataset even when the predicted cluster number differs from the actual cluster number.

When the maximum cluster numbers (k) was set to 20, the fuzziness parameter (m) was set to 2 (default), and the maximum number of iterations (l) was set to 100 (default), the AFCM-JS method estimated the number of clusters as 6 for the synthetic dataset presented in Fig. 3. Fig. 4 displays these clusters obtained by the AFCM-JS and FCM algorithms, along with their centroids indicated by red dots. In the AFCM-JS method, the FCM method is initially run with the maximum cluster number to find clusters and their centroids. As a result of the FCM method's operation, a partition matrix containing the membership of each cluster is obtained. Then, the JS distance matrix is calculated by finding the JS interest between the memberships of each cluster obtained. The appropriate JS threshold value is determined by applying the quartile range method to the data in this JS distance matrix. Cluster merging is performed until the JS threshold value determined by the algorithm is smaller than the minimum JS threshold value. During this merging phase, elements of clusters numbered 0, 2, 3, 5, 8, 9, 10, 12, 14, 15, 16, 17, 18, 19 were included in cluster centers numbered 1, 4, 6, 7, 11, 13.

The FCM method is already a technique used for clustering and finding cluster centers. In terms of its ability to determine clusters and cluster centers, the AFCM-JS method, with the same number of clusters, is also expected to find cluster centers similar to the FCM method. To compare the performance of the proposed AFCM-JS method, a comparison of the cluster centers of the two methods provided in Fig. 3 is presented with numerical values in Table 2. Upon examining Table 2, it can be observed that the cluster centers of the AFCM-JS and FCM methods are similar to each other.

The results of the comparison of the AFCM-JS method with other methods are presented in Fig. 5. In this comparison, values obtained by averaging the results of 30 experiments are provided. According to

Table 2
Cluster centers for AFCM-JS and FCM algorithms on synthetic data.

Cluster centers	AFCM-JS		FCM	
	x-pos	y-pos	x-pos	y-pos
Center-0	0.2048	0.1939	0.2072	0.1936
Center-1	0.2000	0.7929	0.1996	0.7913
Center-2	0.2062	0.4988	0.2090	0.5005
Center-3	0.6839	0.2106	0.6987	0.2279
Center-4	0.8140	0.5086	0.8183	0.5149
Center-5	0.9126	0.9195	0.9089	0.9129

Table 3
Comparison of cluster validity indices for FCM, AFCM-JS, PCM, and PFCM.

Validity index	AFCM-JS	FCM	PCM	PFCM
PE	0.2734	0.2745	0.741422	0.6138
PC	0.7129	0.7112	0.23674	0.3227
MPE	0.2751	0.2762	0.744590	0.6101
MPC	0.6555	0.6534	0.19240	0.2025
XB	0.09379	0.09561	1903827.4098	0.5757
SC	1409.5734	1394.5061	12.9017	102.7029
FS	-104.2678	-103.46	13.4861	-19.0400

the results presented in Fig. 5, AFCM-JS achieved the highest accuracy percentage with 81.9366%. Following that, FCM obtained the second-highest accuracy with a rate of 81.7059%. AFCM-JS outperformed other methods with this approach, where the number of clusters is unknown, and the threshold value is determined by the algorithm. In Table 3, the AFCM-JS method was compared with the FCM, PCM, and PFCM methods using 7 different cluster validity indices based on the average obtained from 30 runs. According to Table 3, when comparing with cluster validity indices that yield better results with increasing values such as PC, MPC, and SC, and those that yield better results with decreasing values such as PE, MPE, XB, and FS, it can be concluded that the AFCM-JS method is successful compared to FCM, PCM, and PFCM methods.

5.3. Movie data set

The MovieLens datasets were published by GroupLens at the University of Minnesota, are mainly used in the experiments [30]. The movie data set includes a total of 9742 movies and their genres. Fig. 6 shows the names of the movies in the data set and the information about which genre each movie belongs to.

To enhance the performance and accuracy of the proposed method in this data set containing 20 different movie genres, 5 distinct movie genres were utilized. When deciding on the selection of these 5 different movie genres, preference was given to commonly used genres, aiming to include the maximum number of films. Thus, a small number of decisive features were included for most movies in the data set. Fig. 7 shows a general schema of the newly created data set. In Fig. 7, the information about the genre included in the movie is marked as 1, while the genre not included is marked as 0. In Fig. 6, the movie 'Toy Story' includes adventure, animation, children, comedy, and fantasy genres, whereas in Fig. 7, the same movie is limited to adventure, animation, and comedy genres. The newly curated movie data set comprises a total of 7650 movies.

To maintain the information about the genres included in each movie in a probability distribution between 0–1 and ensure that the sum of the attributes is equal to 1, a Dirichlet distribution was used in the data set. The Dirichlet distribution is a generalized form of the beta distribution. This distribution is used to create an efficient model by flexibly matching the data according to the weights of the attributes [31]. The final form of the data set after weighting with Dirichlet distribution of the movie data set consisting of 0's and 1's is shown in Fig. 8. For example, the Toy Story movie has values close to 1 for the animation, adventure, and comedy genres it contains,

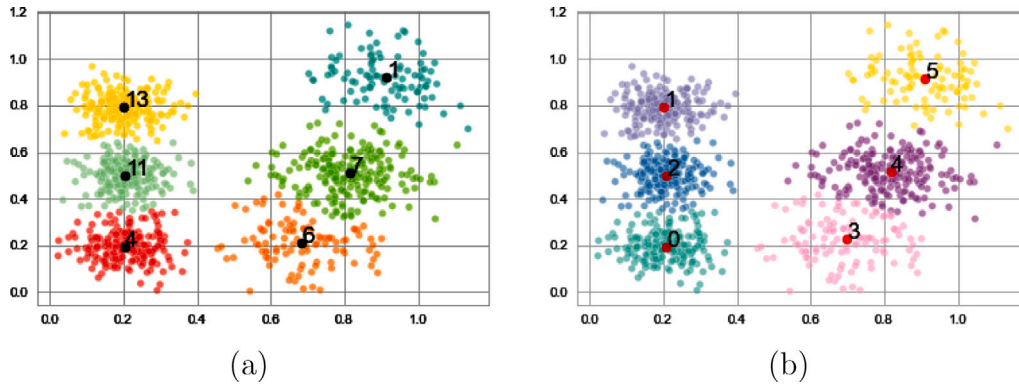


Fig. 4. Clustering of Synthetic Data Set. (a) AFCM-JS (b) FCM.

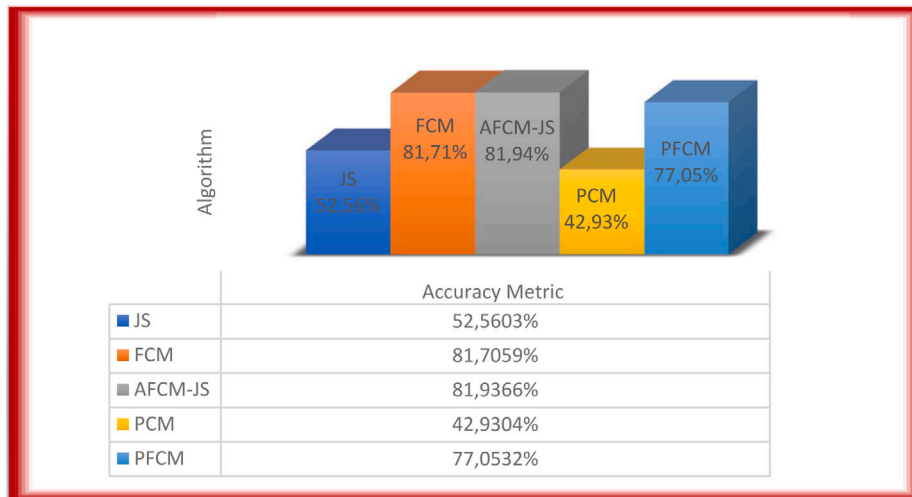


Fig. 5. Algorithms and accuracy metric.

Movielid	Title	Genres
1	Toy Story (1995)	[Adventure, Animation, Children, Comedy, Fantasy]
2	Jumanji (1995)	[Adventure, Children, Fantasy]
3	Grumpier Old Men (1995)	[Comedy, Romance]
4	Waiting to Exhale (1995)	[Comedy, Drama, Romance]
5	Father of the Bride Part II (1995)	[Comedy]

Fig. 6. The actual appearance of the used movie data.

Movielid	Title	Genres
1	Toy Story (1995)	[Adventure, Animation, Children, Comedy, Fantasy]
2	Jumanji (1995)	[Adventure, Children, Fantasy]
3	Grumpier Old Men (1995)	[Comedy, Romance]
4	Waiting to Exhale (1995)	[Comedy, Drama, Romance]
5	Father of the Bride Part II (1995)	[Comedy]

Fig. 7. The feature schema of the new movie data.

while the drama and horror genres that are not included in the movies genre are weighted close to 0. The total weight of a movie's entire distributed weight is equal to 1. Examining Table 4 reveals that the AFCM-JS method demonstrates stability in cluster predictions for the movie dataset under different maximum cluster numbers, similar to its behavior with the synthetic dataset. When the AFCM-JS algorithm

was tested with varying maximum cluster numbers under the same initial conditions for the movie dataset, the estimated cluster numbers remained unchanged. When the maximum cluster numbers (k) was set to 25, the fuzziness parameter (m) was set to 2 (default), and the maximum number of iterations (l) was set to 100 (default), the AFCM-JS method estimated the number of clusters as 14. The results

Movielid	Title	Adventure	Animation	Comedy	Drama	Horror
1	Toy Story (1995)	1	1	1	0	0
2	Jumanji (1995)	1	0	0	0	0
3	Grumpier Old Men (1995)	0	0	1	0	0
4	Waiting to Exhale (1995)	0	0	1	1	0
5	Father of the Bride Part II (1995)	0	0	1	0	0

Fig. 8. The final view of the data set after applying Dirichlet distribution.

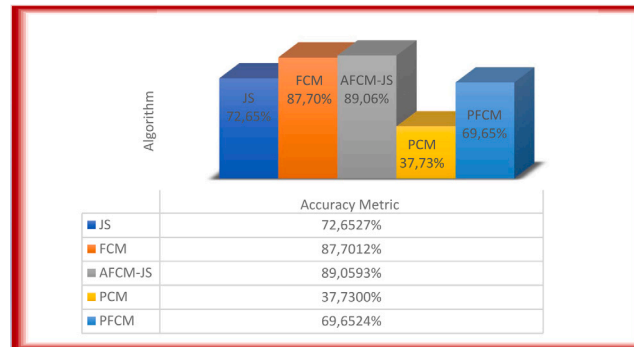


Fig. 9. Algorithms and accuracy metric.

Table 4

Estimated cluster numbers with the AFCM-JS method for the movie data.

Maximum cluster numbers	Estimated cluster numbers
25	14
31	14
40	14
50	14

Table 5

Comparison of cluster validity indices for AFCM-JS, FCM, PCM, PFCM.

Validity index	AFCM-JS	FCM	PCM	PFCM
PE	0.3782	0.4542	0.9658	0.9522
PC	0.6845	0.5405	0.1400	0.1501
MPE	0.3789	0.4550	0.9662	0.9585
MPC	0.6602	0.5052	0.1377	0.1455
XB	0.0109	0.03519	181.6600	169.02427
SC	7150.0802	7149.8883	35.9880	66.6402
FS	-2158.4487	-1568.9578	161.6901	-127.2987

of comparing the AFCM-JS method with other methods for the movie dataset are presented in Fig. 9. According to the results shown in Fig. 9, the AFCM-JS method achieved the highest accuracy rate, reaching 89.0593%.

By obtaining higher accuracy values in clustering results compared to both the JS and FCM methods, the AFCM-JS method integrates the advantages of both approaches. Some cluster validity indices may exhibit a monotonous decrease as the number of clusters increases. Additionally, cluster validity indices evaluate cluster formations based on different criteria, such as separability, compactness, or a combination of both. Therefore, to validate the accuracy of the proposed method, widely used performance indices in fuzzy clustering methods such as PE, PC, MPE, MPC, FS, SCG, and XB were employed. The PC, MPC, and SC indices indicate more successful clustering with increasing values, whereas the PE, MPE, XB, and FS cluster validity indices suggest more successful clustering with decreasing values. When examining Table 5, it is evident that the AFCM-JS method outperforms other methods across all seven cluster validity indices.

6. Conclusion and discussion

In this study, a novel fuzzy and interest-based clustering method is proposed. This method combines the FCM algorithm with the JS distance metric to estimate the appropriate number of clusters when the exact number is unknown. The AFCM-JS method is designed to use the interquartile range for determining the threshold value of the JS parameter, enabling the algorithm to dynamically adjust the threshold through an adaptive threshold determination strategy. To evaluate the clustering performance of AFCM-JS method, an synthetic dataset consisting of 6 clusters was created. It was observed AFCM-JS method correctly estimated the number of clusters. The equivalence of cluster centers between the proposed methods and the FCM method validates the success of the methods. AFCM-JS method were compared with FCM, PFCM, PCM, and JS clustering results. When the values obtained from the accuracy metric and 7 different cluster validity indices were examined, it was concluded that the AFCM-JS method was successful. The AFCM-JS method was applied to a movie data set to recommend similar films. When the AFCM-JS method was tested with different maximum cluster numbers under the same initial conditions, it was observed that the algorithm predicted similar cluster numbers. When examining the membership values in the genre distributions of these cluster groups for any film, the method's performance was observed. The FCM algorithm does not consider the interest between the data points and creates as many clusters as the given number of clusters. The JS algorithm is interest-based method and does not calculate cluster centers. AFCM-JS method overcomes the limitations of FCM, allowing clustering according to the desired number of clusters, and addresses the absence of a mechanism to determine cluster centers in JS-based clustering. Therefore, the AFCM-JS method has been found to be effective in finding the optimal cluster number without knowing the number of clusters, particularly in a movie recommendation system where interest relationships are essential.

The application of the AFCM-JS algorithm to a movie recommendation system demonstrates that the algorithm presents an alternative method for real-world applications. The AFCM-JS method's ability to perform dynamic, interest-based clustering and accurately predict the number of clusters can lead to more accurate, personalized, and time-adaptive results in recommendation systems. This could contribute to improving user satisfaction and enhancing the accuracy of recommendations, while also serving as a precursor to exploring the algorithm's

potential applications in other domains. For instance, the algorithm could be utilized for video search and replication, personalized product recommendations on e-commerce platforms, patient-specific treatment recommendations in healthcare systems, or content personalization on social media platforms. Investigating these possibilities emphasizes the versatility and robustness of the AFCM-JS algorithm, broadening its utility across various industries and potentially sparking greater interest in the development of alternative methods. In future studies, should broader datasets suitable for the method be produced, extending these analyses would certainly be considered.

CRediT authorship contribution statement

Gökhan Kayhan: Writing – review & editing, Writing – original draft, Supervision, Software, Methodology. **Naciye Aydin:** Writing – review & editing, Writing – original draft, Supervision, Software, Methodology. **Sercan Demirci:** Writing – review & editing, Writing – original draft, Supervision, Software, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] R. Duan, C. Jiang, H.K. Jain, Combining review-based collaborative filtering and matrix factorization: A solution to rating's sparsity problem, *Decis. Support Syst.* 156 (2022) 113748.
- [2] P. Kumar, R.S. Thakur, Recommendation system techniques and related issues: a survey, *Int. J. Inf. Technol.* 10 (2018) 495–501.
- [3] J. Bobadilla, F. Ortega, A. Hernando, A. Gutiérrez, Recommender systems survey, *Knowl.-Based Syst.* 46 (2013) 109–132.
- [4] G. Adomavicius, A. Tuzhilin, Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions, *IEEE Trans. Knowl. Data Eng.* 17 (6) (2005) 734–749.
- [5] J. Son, S.B. Kim, Content-based filtering for recommendation systems using multiattribute networks, *Expert Syst. Appl.* 89 (2017) 404–412.
- [6] A. Gazdar, L. Hidri, A new similarity measure for collaborative filtering based recommender systems, *Knowl.-Based Syst.* 188 (2020) 105058.
- [7] Z. Cui, X. Xu, X. Fei, X. Cai, Y. Cao, W. Zhang, J. Chen, Personalized recommendation system based on collaborative filtering for IoT scenarios, *IEEE Trans. Serv. Comput.* 13 (4) (2020) 685–695.
- [8] B. Walek, V. Fojtik, A hybrid recommender system for recommending relevant movies using an expert system, *Expert Syst. Appl.* 158 (2020) 113452.
- [9] M. Aggarwal, S. Goel, P.K. Aggarwal, T. Srivastava, A. Srivastava, V. Gupta, I. Ishani, A recommendation system to envision movie ranking using collaborative filtering, in: 2023 International Conference on Artificial Intelligence and Smart Communication, AISC, IEEE, 2023, pp. 755–758.
- [10] S. Zahra, M.A. Ghazanfar, A. Khalid, M.A. Azam, U. Naeem, A. Prugel-Bennett, Novel centroid selection approaches for KMeans-clustering based recommender systems, *Inform. Sci.* 320 (2015) 156–189.
- [11] N. Vara, M. Mirzabeigi, H. Sotudeh, et al., Application of k-means clustering algorithm to improve effectiveness of the results recommended by journal recommender system, *Scientometrics* 127 (2022) 3237–3252.
- [12] S. Mann, S. Chawla, A proposed hybrid clustering algorithm using K-means and BIRCH for cluster based cab recommender system (CBCRS), *Int. J. Inf. Technol.* 15 (2023) 219–227.
- [13] Q. Li, B.M. Kim, Clustering approach for hybrid recommender system, in: Proceedings IEEE/WIC International Conference on Web Intelligence (WI 2003), 2003, pp. 33–38.
- [14] Y. Gulzar, A.A. Alwan, R.M. Abdullah, A.Z. Abualkashik, M. Oumrani, OCA: Ordered clustering-based algorithm for E-commerce recommendation system, *Sustainability* 15 (4) (2023).
- [15] G. Guo, J. Zhang, N. Yorke-Smith, Leveraging multiviews of trust and similarity to enhance clustering-based recommender systems, *Knowl.-Based Syst.* 74 (2015) 14–27.
- [16] O. Khoo, Picturing diversity: Netflix's inclusion strategy and the Netflix Recommender Algorithm (NRA), *Telev. & New Media* 24 (3) (2023) 281–297.
- [17] R. Rashidi, K. Khamforoosh, A. Sheikahmadi, Proposing improved meta-heuristic algorithms for clustering and separating users in the recommender systems, *Electron. Commer. Res.* (2022) 1–26.
- [18] H. Koochi, K. Kiani, User based collaborative filtering using fuzzy C-means, *Measurement* 91 (2016) 134–139.
- [19] J.C. Bezdek, R. Ehrlich, W. Full, FCM: The fuzzy c-means clustering algorithm, *Comput. Geosci.* 10 (2–3) (1984) 191–203.
- [20] J. Lin, Divergence measures based on the Shannon entropy, *IEEE Trans. Inform. Theory* 37 (1) (1991) 145–151.
- [21] J. Yang, S. Rahardja, P. Fränti, Outlier detection: how to threshold outlier scores? in: Proceedings of the International Conference on Artificial Intelligence, Information Processing and Cloud Computing, 2019, pp. 1–6.
- [22] M. Ozer, Fuzzy c-means clustering and internet portals: A case study, *European J. Oper. Res.* 164 (3) (2005) 696–714.
- [23] J.C. Bezdek, Cluster validity with fuzzy sets, *J. Cybern.* 3 (3) (1973) 58–73.
- [24] J. Bezdek, Numerical taxonomy with fuzzy sets, *J. Math. Biol.* 1 (1974) 57–71, <http://dx.doi.org/10.1007/BF02339490>.
- [25] K. Schenatto, E.G. De Souza, C.L. Bazzi, A. Gavioli, N.M. Betzek, H.M. Beneduzzi, Normalization of data for delineating management zones, *Comput. Electron. Agric.* 143 (2017) 238–248.
- [26] B. Boydell, A. McBratney, Identifying potential within-field management zones from cotton-yield estimates, *Precis. Agric.* 3 (2002) 9–23.
- [27] S.-B. Cho, S.-H. Yoo, Fuzzy Bayesian validation for cluster analysis of yeast cell-cycle data, *Pattern Recognit.* 39 (12) (2006) 2405–2414.
- [28] M. Bouguessa, S. Wang, H. Sun, An objective approach to cluster validation, *Pattern Recognit. Lett.* 27 (13) (2006) 1419–1430.
- [29] X. Zhang, J. Liu, Q. Zhang, W. Zhu, gMeasure: A group-based network performance measurement service for peer-to-peer applications, in: Global Telecommunications Conference, 2002. GLOBECOM'02. IEEE, vol. 3, IEEE, 2002, pp. 2528–2532.
- [30] GroupLens, University of Minnesota through the MovieLens, <https://movielens.org/> (Accessed 05 October 2024).
- [31] A. Nedaie, A.A. Najafi, Support vector machine with Dirichlet feature mapping, *Neural Netw.* 98 (2018) 87–101.